

Bankers algorithm

241401084

```
#include <stdio.h>

int main()
{
    int n, r, i, j, k;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter number of resource types: ");
    scanf("%d", &r);

    int alloc[n][r], max[n][r], need[n][r], avail[r];
    int finish[n], safe_sequence[n];
    int count = 0, safeFound;

    // Allocation Matrix
    printf("Enter Allocation Matrix:\n");
    for(i = 0; i < n; i++)
    {
        for(j = 0; j < r; j++)
        {
            scanf("%d", &alloc[i][j]);
        }
    }

    // Max Matrix
    printf("Enter Max Matrix:\n");
    for(i = 0; i < n; i++)
    {
        for(j = 0; j < r; j++)
        {
            scanf("%d", &max[i][j]);
        }
    }

    // Available Resources
    printf("Enter Available Resources:\n");
    for(i = 0; i < r; i++)
    {
        scanf("%d", &avail[i]);
    }

    // Initialize finish array
    for(i = 0; i < n; i++)
    {
        finish[i] = 0;
    }

    -- INSERT --
```

1,

```

        scanf("%d", &avail[i]);
    }

    // Initialize finish array
    for(i = 0; i < n; i++)
    {
        finish[i] = 0;
    }

    // Calculate Need Matrix
    for(i = 0; i < n; i++)
    {
        for(j = 0; j < r; j++)
        {
            need[i][j] = max[i][j] - alloc[i][j];
        }
    }

    // Banker's Algorithm
    while(count < n)
    {
        safeFound = 0;

        for(i = 0; i < n; i++)
        {
            if(finish[i] == 0)
            {
                int safe = 1;

                for(j = 0; j < r; j++)
                {
                    if(need[i][j] > avail[j])
                    {
                        safe = 0;
                        break;
                    }
                }

                if(safe)
                {
                    for(k = 0; k < r; k++)
                    {
                        avail[k] += alloc[i][k];
                    }
                }
            }
        }
    }
}

-- INSERT --

```

```

        if(finish[i] == 0)
        {
            int safe = 1;

            for(j = 0; j < r; j++)
            {
                if(need[i][j] > avail[j])
                {
                    safe = 0;
                    break;
                }
            }

            if(safe)
            {
                for(k = 0; k < r; k++)
                {
                    avail[k] += alloc[i][k];
                }

                safe_sequence[count] = i;
                finish[i] = 1;
                count++;
                safeFound = 1;
            }
        }
    }

    if(!safeFound)
    {
        printf("System is in unsafe state! Deadlock may occur.\n");
        return 0;
    }
}

// Print Safe Sequence
printf("Safe sequence is: ");
for(i = 0; i < n; i++)
{
    printf("P%d ", safe_sequence[i]);
}

return 0;
}

-- INSERT --

```

110,1

output:-

```
[user123@localhost ~]$ ls
0          backup_test  dinesh    green.txt  pc          sharmi.c   Videos
abi.c      banker             dir        input.sh   Pictures    sharmila.sh
abinaya.c  blue.sh           Documents  leap.sh    pink.sh     sharm.sh
ab.sh      blue.txt          Downloads  mango.txt  Public      sharmz
a.out      Desktop          file.txt   Music      red.txt     Templates
[user123@localhost ~]$ mkdir bankers
[user123@localhost ~]$ cd bankers
[user123@localhost bankers]$ vi bankers.c
[user123@localhost bankers]$ gcc bankers.c
[user123@localhost bankers]$ ./a.out
Enter number of processes: 4
Enter number of resource types: 3
Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
2 1 1
Enter Max Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
Enter Available Resources:
3 3 2
Safe sequence is: P1 P3 P0 P2 [user123@localhost bankers]$
```